ESTRUTURANDO SCRIPTS GRANDES E BIBLIOTECAS

*O `deploy.sh` tinha 1.400 linhas. Ninguém o escreveu: ele **cresceu** — trinta linhas em 2021 e um trecho novo a cada incidente, sempre colado no fim, sempre por alguém com pressa. Quatro funções de log com nomes diferentes, dezenove `curl` ligeiramente distintos, e no meio uma linha que ninguém ousava tocar: `# NAO REMOVER -- quebra a producao (nao sabemos por que)`. **Cada acréscimo isolado era razoável; ninguém parou para reorganizar.***

077
GUIAS LINUX

AS OITO CAMADAS DE UM SCRIPT SÉRIO · E A DIREÇÃO DA DEPENDÊNCIA

1	`shebang + cabeçalho de uso`	a ajuda sai daqui por `sed` — nunca dessincroniza
2	`set -Eeuo pipefail` `shopt -s inherit_errexit`	antes de qualquer comando que possa falhar — nem a leitura da biblioteca escapa
3	`constantes readonly`	caminhos, códigos de saída, versão. Atribuição acidental vira erro
4	`estado global mutável`	o mínimo possível: tudo o mais é `local` dentro de função
5	`source das bibliotecas`	**depois** das constantes — a biblioteca pode usá-las
6	`funções: genéricas → específicas`	uma responsabilidade cada, nome `verbo_substantivo`
7	`principal()`	só orquestra; não contém lógica de negócio
8	`guarda do ponto de entrada`	`[[ "${BASH_SOURCE[0]}" == "$0" ]]` — é o que torna o arquivo testável

`bin/`
`implantar.sh`
`reverter.sh`
`coletar.sh`
executáveis

`source` →

`lib/`
`log.sh`
`http.sh`
`validacao.sh`
só definem funções

`tests/`
`log.bats` · `validacao.bats`
carregam a lib e executam o bin com `run`

A dependência aponta numa direção só: `bin/` usa `lib/`; `lib/` não conhece `bin/`. Uma biblioteca que chama função definida no executável cria um **ciclo** que impede testar qualquer um dos dois isoladamente — e é o começo do arquivo de 1.400 linhas, porque a única forma de fazer funcionar passa a ser juntar tudo.

A ordem das camadas não é estética: **cada uma depende só das anteriores.** E o `RAIZ` merece atenção própria — `RAIZ="$(cd -- "$(dirname -- "$(readlink -f -- "${BASH_SOURCE[0]}")")/.." && pwd)"`. O `BASH_SOURCE` é o caminho **do arquivo**, não o do processo, então funciona igual quando o script é chamado por caminho relativo, por link simbólico no `PATH` ou por outro script; o `readlink -f` resolve o link. Usar `$0` ou `$PWD` aqui é o defeito que só aparece quando alguém chama de outro diretório — e aí a mensagem é obscura. **As cinco propriedades de uma biblioteca bem comportada:** (1) **não executa nada** ao ser carregada — `source` que dispara ação é armadilha, porque quem carrega não espera efeito colateral; (2) protege-se contra **carregamento duplo** com `[[ -n "${_LIB_X_CARREGADA:-}" ]] && return 0`, sem o que duas bibliotecas que carregam uma terceira redefinem tudo e o `readonly` explode; (3) **prefixa os nomes** (`log_info`, `http_get`), porque o Bash tem um espaço de nomes só; (4) **não tem shebang**, e sim `# shellcheck shell=bash` para o analisador saber o dialeto; (5) é carregada por **caminho absoluto calculado**, com verificação explícita de existência.

FUNÇÕES, BIBLIOTECAS E CONFIGURAÇÃO NA PRÁTICA

kali@kali: ~/lab-estrutura

```
# a guarda que faz o mesmo arquivo ser programa E biblioteca
$ if [[ "${BASH_SOURCE[0]}" == "${0}" ]]; then principal "$@"; fi
# a ajuda sai do próprio cabeçalho — nunca dessincroniza
$ uso() { sed -n '2,9p' "${BASH_SOURCE[0]}" | sed 's/^# \?//'; }
# carregar com mensagem clara em vez de erro obscuro do set -e
$ carregar() {
    local arq="$RAIZ/lib/$1"
    [[ -r "$arq" ]] || { printf '%s: biblioteca ausente: %s\n' \
        "$PROGRAMA" "$arq" >&2; exit 70; }
    # shellcheck source=/dev/null
    source "$arq"
  }
$ head -3 lib/log.sh
# shellcheck shell=bash
[[ -n "${_LIB_LOG_CARREGADA:-}" ]] && return 0
readonly _LIB_LOG_CARREGADA=1
# devolver ESTRUTURA sem subshell e sem serializar em texto
$ carregar_config() { local -n _ref=$1; _ref[host]='exemplo.local'; }
$ declare -A cfg; carregar_config cfg; echo "${cfg[host]}"
exemplo.local
# o teste carrega a lib e executa o bin — a seta da figura, na prática
$ bats tests/
6 tests, 0 failures
```

FUNÇÕES: AS REGRAS

`local`	**toda** variável de dentro. Sem isso o `i` do laço interno destrói o `i` do chamador, e o sintoma aparece **longe da causa**. `local -r` constante, `-a` array, `-i` inteiro
`_prefixo`	função interna de biblioteca — sinaliza que não é interface pública
`status`	devolver **sim ou não**: `eh_valido "$v" || die …`
`stdout`	devolver **um valor**, capturado com `$( )`
`local -n`	devolver **estrutura**: cria referência ao array do chamador, sem `fork`. Prefixe o parâmetro (`_ref`) ou colide com o nome de fora
`contrato`	três linhas de comentário: assinatura, o que faz, e **quais status devolve**

CONFIGURAÇÃO: DA MENOR À MAIOR PRIORIDADE

1. padrões no próprio script
2. `/etc/meuprog/config`
3. `${XDG_CONFIG_HOME:-$HOME/.config}/meuprog/config`
4. variáveis de ambiente (`MEUPROG_DESTINO`)
5. **opções de linha de comando** — analisadas por último, sobrepõem tudo

Saiba quando parar de usar shell — é o ponto que teria salvado o `deploy.sh`. O shell é excelente em orquestrar processos, encadear ferramentas e manipular arquivos, e ruim em tudo o mais. Os sinais de que o limite foi ultrapassado: **estrutura de dados aninhada** (simular objeto dentro de lista com nomes de variáveis concatenados é o sinal mais claro de todos); **aritmética não inteira**, que obriga a chamar `bc` ou `awk` a cada operação; **tratamento de erro por item** com retentativa e registro, que sai com três vezes o tamanho do equivalente em Python; **concorrência com estado compartilhado**; **passar de 300 ou 400 linhas** sem que a maior parte seja invocação de comando externo; e **precisar de testes de unidade extensos**. A migração não é total: mantenha o shell como **casca** — analisar opções, verificar dependências, orquestrar — e delegue a lógica com `exec python3 "$RAIZ/lib/processar.py" --ambiente "$AMBIENTE"`. O `exec` **substitui** o processo do shell, então sinais e códigos de saída se propagam sem uma camada no meio. **E se você herdou o arquivo de 1.400 linhas, refatorar vence reescrever**, nesta ordem: (1) controle de versão; (2) `shellcheck`, corrigindo só a severidade alta; (3) acrescentar `set -Eeuo pipefail` e rodar — **cada quebra é um bug que já existia e estava sendo engolido**; (4) escrever um **teste de caracterização** em `bats` que registre a saída atual, correta ou não, como rede de segurança; (5) extrair funções **uma por vez**, rodando o teste a cada extração; (6) só então unificar as quatro funções de log e descobrir o que o `sleep 3` escondia (quase sempre uma condição de corrida que deveria ser um laço de espera com verificação). **Reescrever do zero não funciona:** o arquivo contém anos de correções de casos particulares que ninguém documentou, e a reescrita as perde todas — para redescobri-las uma a uma, em produção.

`source` de arquivo de configuração é execução de código arbitrário. Um `config` gravável por outro usuário — ou que veio do cliente por e-mail — roda **com o privilégio de quem executa o script**, no instante do `source`, e nada no arquivo precisa parecer suspeito para isso acontecer. Se a origem não é totalmente confiável, ou verifique dono e permissão antes — `[[ "$(stat -c '%U %a' "$arq")" == "root 600" ]] || die 70 "config com permissao insegura: $arq"` — ou **leia como dados**, com `jq` ou um laço de `chave=valor` validado, em vez de `source`. Pelo mesmo motivo, marque os caminhos de saída como `readonly` logo depois da análise de opções: com `SAIDA` mutável, um trecho posterior pode sobrescrevê-la e o relatório vai parar no lugar errado — ou **por cima do de outro alvo**.

RESUMO RÁPIDO

- ✓ **Script grande tem forma reconhecível:** cabeçalho de uso, `set -Eeuo pipefail`, constantes `readonly`, estado mínimo, `source` das bibliotecas, funções, `principal()` e a guarda do ponto de entrada.
- ✓ `RAIZ` **calculado a partir de** `BASH_SOURCE`, não de `$0` nem de `pwd` — e com `readlink -f` se o script puder ser chamado por link simbólico.
- ✓ **A guarda** `[[ "${BASH_SOURCE[0]}" == "$0" ]]` **é o que torna o arquivo testável:** executado roda o programa, carregado só define as funções.
- ✓ **Toda variável de função é** `local`**.** Sem isso o `i` de dentro destrói o `i` de fora, e o sintoma aparece longe da causa.
- ✓ **Três formas de devolver valor:** status para sim ou não, `stdout` capturado para um valor, `local -n` para estrutura sem subshell.
- ✓ **Biblioteca não executa nada ao ser carregada**, protege-se contra carregamento duplo, prefixa os nomes e é carregada por caminho absoluto com verificação explícita.
- ✓ **A dependência aponta numa direção só:** `bin/` usa `lib/`, nunca o contrário. O ciclo é o começo do arquivo de 1.400 linhas.
- ✓ **Saiba quando parar:** estrutura aninhada, ponto flutuante, erro por item, mais de 300 linhas de lógica própria. A saída é o shell como casca e o motor em outra linguagem, entregue com `exec`.
- ✓ **Refatorar vence reescrever** — versão, `shellcheck`, `set -Eeuo pipefail`, teste de caracterização, extração função a função, nessa ordem.

NO KALI

A caixa de ferramentas da equipe é uma biblioteca, não vinte scripts com o mesmo `log()` **copiado.** Padronizar `lib/log.sh`, `lib/escopo.sh` e `lib/relatorio.sh` faz com que a verificação de escopo e o formato do relatório sejam idênticos em todo utilitário — e uma correção na verificação passa a valer **para todos de uma vez**, em vez de ser replicada em vinte arquivos, onde inevitavelmente um fica para trás. **A validação de escopo pertence à biblioteca e é chamada por todo executável:** uma função `escopo_validar "$alvo" || die "$EX_USO" "fora do escopo"` centralizada e testada com `bats` contra casos de borda (CIDR, nome de host, IPv6) é o que evita que um utilitário novo escrito às pressas dispare contra alvo não autorizado — o que caracteriza acesso não autorizado nos termos da Lei 12.737/2012 **independentemente de haver contrato para os demais alvos.** `source` **de configuração é execução de código:** arquivo que veio do cliente por e-mail, ou que está em diretório gravável por outros, executa com o seu privilégio — verifique dono e permissão, ou leia como dados. `readonly` **nos caminhos de saída** num utilitário que grava evidência: deixar `SAIDA` mutável permite que um trecho posterior a sobrescreva e o relatório vá para o lugar errado, ou por cima do de outro alvo. **E a divisão que funciona é casca de shell, motor em Python:** correlacionar resultados de varredura, deduplicar achados e gerar relatório são exatamente o trabalho com estrutura aninhada que o shell faz mal — shell para invocar as ferramentas e organizar os arquivos, Python para ler os resultados e produzir o documento final. **Família RHEL:** a estrutura de projeto independe de distribuição, e `/usr/local/bin` + `/usr/local/lib/<programa>/` + `/etc/<programa>/` valem nas duas famílias — mas com SELinux o executável fora de `/usr/bin` precisa do rótulo correto (`restorecon -Rv /usr/local/bin`), sob pena de funcionar chamado à mão e falhar quando invocado por um serviço. Nas duas famílias, empacotar como `.deb` ou `.rpm` é preferível a copiar arquivos: o gerenciador registra o que está instalado e permite remover sem deixar resto.